

Task 9: Research Activity

A Study of Activation Functions in Neural Networks

Formulas, Graphs, Origins, Trade-offs, and Applications

Introduction

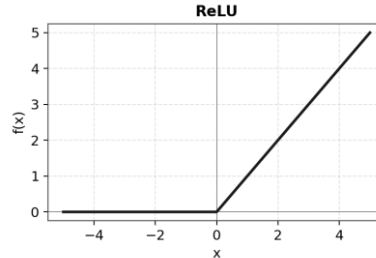
Activation functions introduce non-linearity into neural networks, allowing them to approximate complex functions rather than behaving as simple linear models. The choice of activation function affects a network's training dynamics, convergence speed, gradient flow, and final performance. This report studies ten widely used activation functions — ReLU, Leaky ReLU, PReLU, ELU, GELU, SELU, Swish, Mish, Softplus, and Softmax — covering each function's formula, graph, inventor and year of introduction, advantages, disadvantages, typical applications, and the original research paper that introduced it.

1. ReLU (Rectified Linear Unit)

Formula

$$f(x) = \max(0, x)$$

Graph



Origin

Inventor(s)	Popularized by Vinod Nair and Geoffrey Hinton (earlier used in other forms by Kunihiro Fukushima and others)
Year Introduced	2010 (popularized for deep learning); earlier roots in the 1960s-1980s

Advantages

- Computationally very cheap — just a thresholding operation
- Reduces the vanishing gradient problem present in sigmoid/tanh for positive inputs
- Induces sparsity since negative inputs are mapped to exactly zero
- Empirically speeds up convergence of stochastic gradient descent

Disadvantages

- "Dying ReLU" problem — neurons can get stuck outputting zero permanently if weights push them into the negative region
- Not zero-centered, which can slow optimization
- Unbounded output can occasionally cause exploding activations

Applications

- Default activation in most convolutional neural networks (CNNs)
- Hidden layers of deep feedforward networks
- Image classification architectures such as AlexNet and VGG

Research Paper Reference

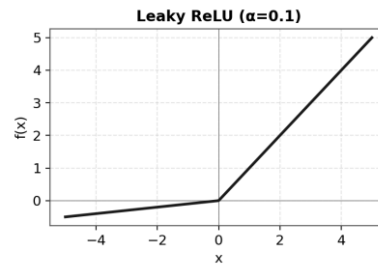
Nair, V., & Hinton, G. E. (2010). *Rectified Linear Units Improve Restricted Boltzmann Machines*. *Proceedings of the 27th International Conference on Machine Learning (ICML)*.

2. Leaky ReLU

Formula

$$f(x) = x, \text{ if } x > 0; f(x) = \alpha x, \text{ if } x \leq 0 \quad (\text{commonly } \alpha = 0.01)$$

Graph



Origin

Inventor(s)	Andrew L. Maas, Awni Y. Hannun, and Andrew Y. Ng
Year Introduced	2013

Advantages

- Allows a small, non-zero gradient when the unit is not active, mitigating the dying ReLU problem
- Retains the computational simplicity of ReLU
- Still promotes some sparsity while avoiding fully dead neurons

Disadvantages

- The slope α is a fixed hyperparameter that must be chosen manually
- Results are inconsistent across tasks — improvement over ReLU is not guaranteed
- Still not zero-centered

Applications

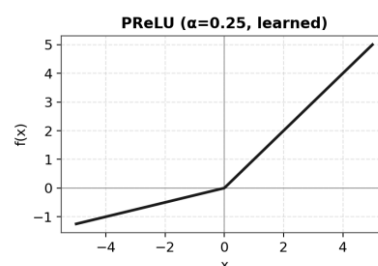
- CNNs and GANs (generative adversarial networks), where dying neurons in the discriminator/generator are especially problematic
- Speech recognition acoustic models

3. PReLU (Parametric ReLU)

Formula

$$f(x) = x, \text{ if } x > 0; f(x) = \alpha x, \text{ if } x \leq 0 \quad (\alpha \text{ is learned during training})$$

Graph



Origin

Inventor(s)	Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun
Year Introduced	2015

Advantages

- The negative-slope parameter α is learned jointly with the network, adapting per channel or layer
- Generalizes both ReLU ($\alpha = 0$) and Leaky ReLU (fixed α) as special cases
- Improved accuracy over ReLU on large-scale image classification in the original study

Disadvantages

- Adds extra trainable parameters, slightly increasing model complexity and overfitting risk on small datasets
- Behaviour depends on careful initialization of α
- Marginal benefit over Leaky ReLU on many practical tasks

Applications

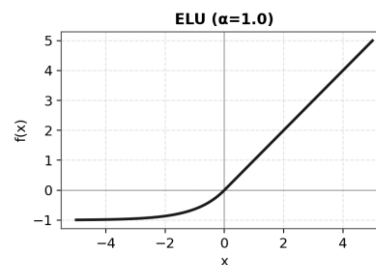
- Deep CNNs for image recognition, notably introduced alongside the ResNet initialization study
- Networks where per-channel adaptive non-linearity improves fine-grained feature learning

4. ELU (Exponential Linear Unit)

Formula

$$f(x) = x, \text{ if } x > 0; f(x) = \alpha(e^x - 1), \text{ if } x \leq 0 \quad (\text{commonly } \alpha = 1)$$

Graph



Origin

Inventor(s)	Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter
Year Introduced	2015

Advantages

- Smooth, continuous curve (including at $x = 0$), which helps gradient-based optimization
- Negative saturation values push mean activations closer to zero, speeding up learning (similar effect to batch normalization)
- Reduces the dying-unit problem seen in ReLU

Disadvantages

- Computationally more expensive than ReLU due to the exponential term
- Can produce large negative outputs for very negative inputs, saturating for large $|x|$
- Requires tuning of the α hyperparameter

Applications

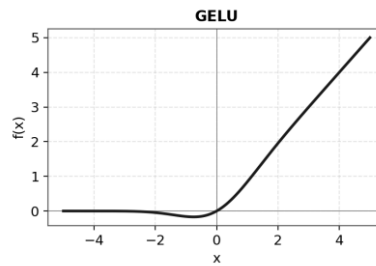
- Deep CNNs where faster convergence and better generalization than ReLU are desired
- Used in some image classification and object detection backbones

5. GELU (Gaussian Error Linear Unit)

Formula

$$f(x) = x \cdot \Phi(x), \text{ where } \Phi(x) \text{ is the standard Gaussian cumulative distribution function (often approximated with tanh)}$$

Graph



Origin

Inventor(s)	Dan Hendrycks and Kevin Gimpel
Year Introduced	2016

Advantages

- Smooth, non-monotonic curve that weights inputs by their percentile under a Gaussian, giving a probabilistic interpretation
- Performs strongly in practice on large Transformer-based models
- Avoids the hard cutoff of ReLU, improving gradient flow near zero

Disadvantages

- More computationally expensive than ReLU (involves an error function or an approximation)
- Less interpretable / harder to reason about than simple piecewise-linear functions

Applications

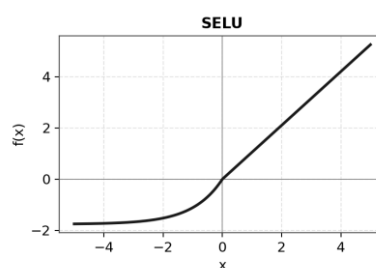
- Standard activation in Transformer architectures such as BERT and GPT-family models
- Vision Transformers (ViT) and other modern large-scale models

6. SELU (Scaled Exponential Linear Unit)

Formula

$$f(x) = \lambda x, \text{ if } x > 0; \quad f(x) = \lambda \alpha (e^x - 1), \text{ if } x \leq 0 \quad (\lambda \approx 1.0507, \alpha \approx 1.6733)$$

Graph



Origin

Inventor(s)	Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter
Year Introduced	2017

Advantages

- Self-normalizing: activations automatically converge toward zero mean and unit variance across layers, given proper (LeCun normal) initialization
- Can remove the need for explicit batch normalization in feedforward networks
- Retains the benefits of ELU (smoothness, non-zero negative outputs)

Disadvantages

- Self-normalizing property only holds under specific conditions — fully connected architecture, LeCun normal weight initialization, and alpha-dropout
- Less effective or reliable in CNNs and architectures with skip connections
- Sensitive to deviations from its required setup

Applications

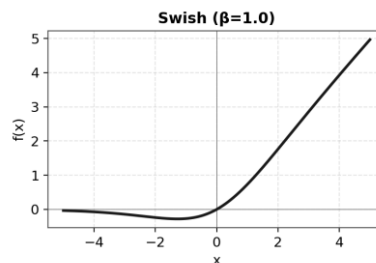
- Feedforward and fully connected deep networks needing self-normalization without batch norm
- Some tabular-data and finance-oriented neural network models

7. Swish

Formula

$f(x) = x \cdot \sigma(\beta x)$, where σ is the sigmoid function (β is fixed or learnable; $\beta = 1$ gives the common SiLU form)

Graph



Origin

Inventor(s)	Prajit Ramachandran, Barret Zoph, and Quoc V. Le (Google Brain)
Year Introduced	2017

Advantages

- Smooth, non-monotonic function discovered via automated search that consistently matched or outperformed ReLU on deep networks
- Unbounded above and bounded below, which helps avoid saturation on the positive side
- Self-gated — the function uses its own value to gate itself, similar in spirit to gating in LSTMs

Disadvantages

- Slightly more computationally expensive than ReLU due to the sigmoid computation
- Benefits are most noticeable in very deep networks; gains are smaller in shallow ones

Applications

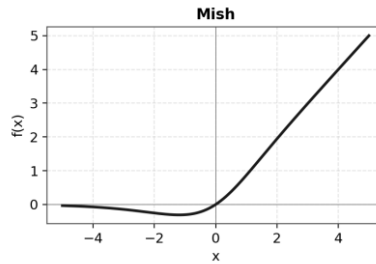
- Mobile-efficient CNN architectures such as EfficientNet and later MobileNet versions
- General deep CNNs where a smooth activation improves training stability

8. Mish

Formula

$$f(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \tanh(\ln(1 + e^x))$$

Graph



Origin

Inventor(s)	Diganta Misra
Year Introduced	2019

Advantages

- Smooth, non-monotonic, and unbounded above — properties associated with better gradient flow than ReLU
- Empirically outperformed ReLU and Swish on several image classification benchmarks in the original study
- Preserves small negative values instead of zeroing them, retaining more information

Disadvantages

- Computationally more expensive than ReLU and even Swish due to the combined softplus and tanh operations
- Slower training throughput on hardware not optimized for these operations

Applications

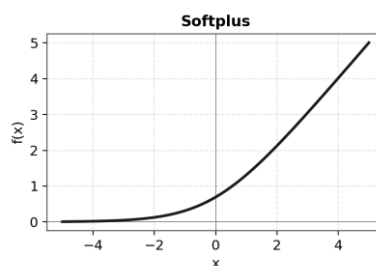
- Object detection models such as YOLOv4 and later YOLO variants
- Image classification backbones seeking marginal accuracy gains over Swish/ReLU

9. Softplus

Formula

$$f(x) = \ln(1 + e^x)$$

Graph



Origin

Inventor(s)	Charles Dugas, Yoshua Bengio, François Bélisle, Claude Nadeau, and René Garcia
Year Introduced	2001

Advantages

- Smooth, differentiable everywhere — acts as a soft, continuous approximation of ReLU
- Always positive output, useful when strictly positive activations are required
- Avoids the hard non-differentiability of ReLU at $x = 0$

Disadvantages

- Computationally more expensive than ReLU (involves exponential and logarithm)
- Never outputs exactly zero, so it does not produce sparse activations
- Can still suffer from vanishing gradients for very negative inputs

Applications

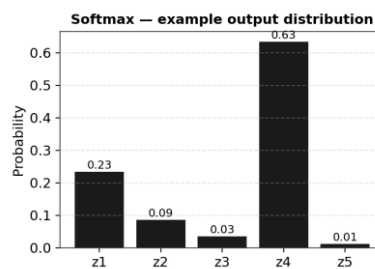
- Used to constrain outputs to be positive, e.g. predicting variance/scale parameters in probabilistic models
- Component of Mish activation (via its softplus term)
- Reinforcement learning policy networks needing smooth positive outputs

10. Softmax

Formula

$$f(x_i) = e^{x_i} / \sum_j e^{x_j} \text{ for } i = 1, \dots, K \text{ classes}$$

Graph



Origin

Inventor(s)	Mathematical origins trace to John S. Bridle, who formalized its use for neural network classification (building on the earlier statistical multinomial logit / Boltzmann distribution)
Year Introduced	1989 (Bridle's formalization for neural networks)

Advantages

- Converts a vector of raw scores (logits) into a valid probability distribution that sums to 1
- Differentiable, enabling gradient-based training with cross-entropy loss
- Naturally highlights the most confident class while still reflecting relative uncertainty among others

Disadvantages

- Sensitive to outlier logits — can saturate and produce near one-hot outputs with vanishing gradients
- Computationally requires exponentials and a normalization sum over all classes, costly for very large class counts
- Assumes classes are mutually exclusive, which is unsuitable for multi-label problems

Applications

- Output layer of multi-class classification networks (image classification, NLP classification)
- Attention mechanisms in Transformers, where softmax normalizes attention scores
- Reinforcement learning policy networks that output action probabilities

Comparative Summary

Across the ten functions studied, a general historical trend emerges: early activations (Softplus, ReLU) prioritized either smoothness or computational simplicity; the 2013-2017 generation (Leaky ReLU, PReLU, ELU, SELU) focused on fixing ReLU's dying-neuron and non-zero-centered issues; and the more recent generation (GELU, Swish, Mish) favors smooth, non-monotonic curves discovered or

motivated by probabilistic reasoning, which have become the default choice in large Transformer and modern CNN architectures. Softmax remains distinct as it operates on a full output vector rather than element-wise, making it the standard choice specifically for the final classification layer.

In practice, ReLU and its variants remain the default for convolutional architectures because of their computational efficiency, while GELU dominates Transformer-based language and vision models, and Swish/Mish appear in architectures where a small accuracy gain justifies the extra computation.